SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO2 ASSESSMENT TOOL 1 – Scenario-Based Requirement Analysis

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 1 – Scenario-Based Requirement Analysis
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	T. Tarun	**Reg. No.:**	1924260001
Date:	27.10.2026	**Duration:**	60 minutes

Code of Conduct
I T. Tarun (1924260001) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the Student: T. Tarun

Annexure A – Scenario-Based Requirement Analysis

Instructions to Students:
Each student will be assigned an individual software project problem statement. Analyze the given scenario and prepare a complete Software Requirement Analysis document by following the steps below.

REAL-TIME WATER MANAGEMENT SYSTEM

1. Study and Analyze the Scenario

1.1 Business Domain and Scenario

The Real-Time Water Management System belongs to the water resource management and smart infrastructure domain. Residential buildings, institutions and municipal facilities require continuous information about tank levels, water flow, pressure and daily consumption. The proposed system combines IoT sensors, a communication network, a backend application and a database to monitor these conditions and display current readings on a web dashboard.

1.2 Existing Challenges

- Manual tank inspection can delay the identification of low water levels and overflow.

●Undetected pipe leakage and unusual consumption can waste water and increase operating costs.

●Irregular monitoring makes it difficult to plan water distribution and maintenance.

●Paper-based records provide limited information for comparing daily or monthly usage.

●Operators may not receive timely warnings when a sensor or water supply component fails.

1.3 Stakeholders

Stakeholder	Role / Responsibility
Water Management Authority	Reviews usage and oversees distribution.
System Administrator	Manages accounts, devices and system configuration.
Maintenance Personnel	Responds to leakage, sensor faults and abnormal pressure.
Consumers / Residents	View water availability and usage notifications.
Facility Manager	Monitors tanks and coordinates water supply.
IoT Sensor / Gateway	Provides readings and communication status.

1.4 System Objectives

●Monitor water level, flow, pressure and consumption at regular intervals.

●Detect potential leaks, overflow and unusual usage patterns.

●Provide timely alerts to responsible users.

●Store historical readings and generate usage reports.

●Support informed water distribution and maintenance decisions.

2. Identify Functional and Non-Functional Requirements

2.1 Functional Requirements

●FR1 – Authentication: The system shall allow registered users to sign in with role-based access.

●FR2 – Sensor Monitoring: The system shall receive tank level, flow and pressure readings from connected devices.

●FR3 – Live Dashboard: The system shall display the latest available readings with timestamps.

●FR4 – Leakage Detection: The system shall flag suspected leakage based on configured flow or consumption rules.

●FR5 – Overflow Warning: The system shall alert users when tank levels exceed a configured threshold.

●FR6 – Low-Level Warning: The system shall notify users when tank water falls below a minimum level.

●FR7 – Usage History: The system shall store timestamped measurements for historical analysis.

●FR8 – Reporting: The system shall generate daily and monthly water usage summaries.

●FR9 – Device Management: Administrators shall register sensors and review their connectivity status.

●FR10 – Alert Management: Authorized users shall acknowledge and review alerts.

2.2 Non-Functional Requirements

●Performance: New readings should appear promptly after successful sensor transmission.

●Security: Authentication, authorization and protected network communication shall be used.

●Reliability: The system shall preserve stored records during temporary connection failures.

●Usability: The dashboard shall present readings and alerts in a clear and understandable format.

●Scalability: The design should support additional tanks, buildings and sensors.

●Maintainability: Sensor interfaces and software components should be independently serviceable.

●Availability: The dashboard should remain accessible during normal operating periods.

●Data Integrity: Readings shall include timestamps and device identifiers to support traceability.

3. Identify Actors and Use Cases

3.1 Primary and Secondary Actors

Actor	Type	Major Interaction
Administrator	Primary	Manage users, sensors and thresholds
Facility Manager	Primary	Monitor readings, review reports and alerts
Maintenance Staff	Primary	Inspect and resolve reported faults
Resident / Consumer	Primary	View permitted usage and notifications
IoT Sensor / Gateway	Secondary	Transmit measurement data
Notification Service	Secondary	Deliver configured alerts

3.2 Major Use Cases

No.	Use Case	Actor
1	Login and view dashboard	Administrator, Facility Manager
2	Register sensor / configure thresholds	Administrator
3	Submit water readings	IoT Sensor / Gateway
4	View tank level and consumption	Facility Manager, Resident
5	Detect leakage / overflow	System

6	Receive and acknowledge alerts	Facility Manager, Maintenance Staff
7	Generate usage report	Facility Manager
8	Update maintenance status	Maintenance Staff

3.3 Use Case Diagram (Text-Based UML Layout)

Administrator → (Manage Users) (Configure Sensors) (Set Alert Thresholds)

IoT Sensor / Gateway → (Submit Water Readings) → (Update Dashboard)

Facility Manager → (View Live Dashboard) (Review Alerts) (Generate Reports)

Maintenance Staff → (Review Fault Alert) (Update Maintenance Status)

Resident → (View Water Usage) (Receive Notifications)

System boundary: All use cases above belong to the Real-Time Water Management System. Each actor connects to the relevant use case; automatic leakage and overflow detection uses the readings supplied by the gateway.

3.4 Use Case Description

When an IoT sensor sends a timestamped water reading, the backend checks the device identifier and stores the data. The system updates the dashboard and compares the measurement against configured limits. If it detects a possible leak, overflow or low level, it records an alert and notifies the facility manager. Maintenance staff investigate the alert and update its status. The manager can later generate a report showing usage and incident history.

4. Prepare the Software Requirement Specification (SRS)

4.1 Introduction

The Real-Time Water Management System is a proposed monitoring platform that helps facilities observe water availability and usage through connected sensors and a centralized interface.

4.2 Problem Description

Conventional water monitoring often depends on periodic manual checks. This may lead to unnoticed tank overflow, delayed leak detection, inaccurate consumption records and slow maintenance response. A real-time system is required to collect measurements, preserve records and inform users about abnormal conditions.

4.3 Scope of the System

The system covers sensor-based monitoring of tank level, flow, pressure and water usage; configurable alerts; user management; historical data storage; and dashboard reporting. It does not directly guarantee water quality, repair damaged pipes or control pumps unless compatible actuators and separate safety controls are installed.

4.4 Functional Requirements

The system shall implement FR1–FR10 described in Section 2.1, including data collection, authentication, threshold-based alerts, device administration, history and reporting.

4.5 Non-Functional Requirements

The system shall satisfy the security, performance, reliability, usability, scalability, maintainability, availability and data-integrity expectations described in Section 2.2.

4.6 Use Case Descriptions

Core workflows include submitting sensor readings, monitoring the dashboard, receiving and acknowledging alerts, reviewing usage history and generating reports. Each workflow requires appropriate permissions and valid sensor data.

4.7 Data Requirements

- User data: user ID, role, login credentials and contact preferences.
- Device data: sensor ID, type, location, calibration information and connectivity state.
- Measurement data: timestamp, water level, flow, pressure and consumption values.
- Alert data: alert ID, type, threshold, severity, timestamp and acknowledgement status.
- Maintenance data: incident ID, assigned staff, actions taken and resolution date.

●Report data: selected date range, usage totals and generated summaries.

4.8 External Interface Requirements

●User Interface: Responsive browser dashboard for live readings, alerts and reports.

●Hardware Interface: Compatible water-level, flow and pressure sensors connected through a gateway.

●Communication Interface: Secure network transfer of readings using an agreed protocol.

●Database Interface: Backend storage and retrieval of measurements, users and alert records.

●Notification Interface: Email, SMS or application alerts when a supported service is configured.

●Export Interface: Downloadable reports such as CSV or PDF.

4.9 System Features

●Real-time tank and pipeline monitoring

●Configurable low-level, overflow and suspected-leak alerts

●Water consumption trends and reports

●Device health and connectivity status

●Role-based dashboards and user accounts

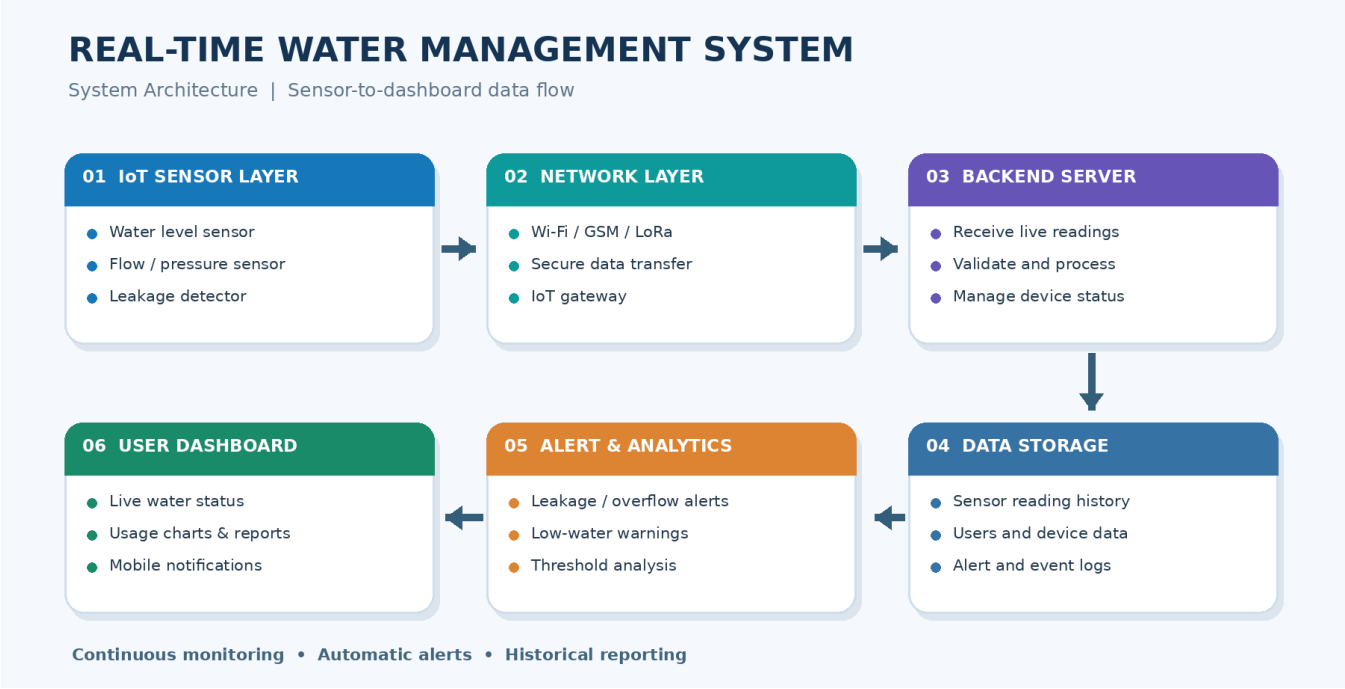
●Alert acknowledgement and maintenance tracking

4.10 System Architecture Diagram

Layer	Flow / Responsibility
Sensing	Water-Level Sensor + Flow Sensor + Pressure Sensor
Connectivity	IoT Gateway → Communication Network
Processing	Backend Server → Threshold Checks and Alert Engine

Storage	Database → Timestamped Measurements and Events
Presentation	Web Dashboard → Reports and Notifications

Architecture flow: IoT Sensors → Gateway / Network → Backend Server → Database + Alert Engine → Dashboard / Notifications.



5. Identify Assumptions and Constraints

5.1 Assumptions

- Sensors are correctly installed and calibrated.
- A power supply and suitable communication network are available.
- Authorized personnel can access the dashboard when required.
- Thresholds are configured for each monitored tank or pipeline.
- Users respond to critical notifications according to facility procedures.

5.2 Constraints

- Technical: Sensor accuracy, power outages and connectivity gaps may affect live monitoring.
- Operational: Devices require periodic inspection, calibration and maintenance.
- Economic: Hardware, data connectivity and maintenance must fit the project budget.
- Legal: Personal information and operational records must be handled according to applicable rules.
- Time: Implementation, integration and testing must fit the academic schedule.

6. Validate Requirement Completeness and Consistency

6.1 Completeness Validation

Verify that every stakeholder need is covered: live readings, historical records, leak and overflow alerts, administration, reporting and maintenance follow-up. Confirm that each monitored device and alert has an identified owner.

6.2 Consistency and Clarity Validation

Use consistent units and timestamps across sensors, database records and dashboard displays. Define alert thresholds explicitly to avoid ambiguity. Ensure that residents cannot modify administrator-only settings.

6.3 Feasibility and Test Validation

- Feed sample readings and verify that the displayed values and timestamps match.
- Simulate low-level, overflow and suspected-leak conditions and check alert creation.
- Disconnect a sensor and verify that stale readings are clearly identified.
- Test access restrictions for administrator, manager, maintenance staff and resident roles.
- Generate reports and compare totals against stored sample measurements.

6.4 Suggested Improvements

Specify measurable refresh intervals, acceptable sensor error, data retention periods and alert delivery targets during implementation planning. Add device-health monitoring and a documented procedure for handling missing or inconsistent sensor data.

7. Conclusion

The proposed Real-Time Water Management System provides a structured approach to monitoring water resources, detecting abnormal conditions and supporting timely maintenance. Its SRS defines the users, data, functions, interfaces, constraints and validation checks needed to guide reliable implementation.

Rubrics for Calculation

Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Scenario Understanding and Problem Analysis (30%)	Demonstrates thorough understanding of the scenario, stakeholders, objectives, and business context with clear analysis.	Demonstrates good understanding with minor omissions.	Basic understanding with limited analysis.	Poor understanding; major aspects missing or incorrect.
Functional and Non-Functional Requirement Identification (25%)	Clearly identifies comprehensive, accurate, and well-classified functional and non-functional requirements.	Identifies most requirements with minor classification errors.	Identifies only basic requirements; several omissions.	Requirements are incomplete, inaccurate, or poorly classified.
Actor Identification and Use Case Modelling (20%)	Correctly identifies all relevant actors and use cases; use case diagram is complete, accurate, and follows UML standards.	Most actors and use cases identified; diagram has minor errors.	Limited actors/use cases identified; diagram partially correct.	Actors/use cases missing or incorrect; diagram absent or inaccurate.
Software Requirement Specification (SRS) Documentation (15%)	SRS is well-structured, complete, professionally organized, and includes all required sections.	SRS is organized with minor missing sections or formatting issues.	SRS includes only essential sections with limited detail.	SRS is incomplete, poorly organized, or lacks essential components.
Assumptions, Constraints, and Requirement Validation (10%)	Clearly identifies realistic assumptions and constraints; validates requirements for completeness, consistency, and ambiguity with appropriate justification.	Identifies most assumptions and constraints; validation is adequate.	Limited assumptions/constraints; validation is superficial.	Assumptions, constraints, and validation are missing or incorrect.

Criteria	**Mark**
Scenario Understanding and Problem Analysis (30%)	/5
Functional and Non-Functional Requirement Identification (25%)	/5
Actor Identification and Use Case Modelling (20%)	/5
Software Requirement Specification (SRS) Documentation (15%)	/5
Assumptions, Constraints, and Requirement Validation (10%)	/5
TOTAL	/25